Product Requirements Document

MARGA: Quantum-Inspired Adaptive Traffic Routing Engine

1. Product Name

MARGA — a quantum-inspired, traffic-adaptive vehicle routing platform for urban, municipal, and logistics fleets.

2. One-Line Description

A software platform that finds low-cost routes on a real road network and re-optimizes them only when traffic becomes meaningfully disruptive, using a quantum-inspired metaheuristic (QPSO), with transparent benchmarking against classical methods.

3. Problem Statement

Urban transportation networks suffer from traffic congestion, inefficient route planning, and high operational costs. The Vehicle Routing Problem (VRP) is NP-hard, so classical exact methods don't scale to city-size fleets, and current quantum hardware can't handle city-scale optimization either. PS 26137 asks for a quantum-inspired metaheuristic framework — using classical hardware but quantum-mechanics-inspired search dynamics (specifically QPSO) — that generates near-optimal routes under real-time or simulated traffic, modeled as a weighted graph, and benchmarked rigorously against conventional metaheuristics and exact methods.

4. Target Users

User	Need
Fleet / logistics operators	Route multiple vehicles efficiently under capacity and time constraints
Municipal transport authorities	Reduce citywide congestion and travel time
Egreen Quanta	Evidence of a working, benchmarked, deployable quantum-inspired system

5. User Pain Points

- Routes planned once at the start of the day go stale as traffic conditions change.
- Exact optimization is computationally infeasible once fleet/stop count grows.
- Existing metaheuristics (GA, ACO, plain PSO) plateau or get stuck in local optima on large, rugged search spaces.
- No visibility into why a route was chosen, or how it compares to simpler methods.
- If the fleet's own vehicles are counted as traffic input, the system can create a feedback loop where its users appear to cause the congestion they are reacting to.
- Re-routing for every small traffic change can create unstable routes, unnecessary computation, and driver confusion.

6. Proposed Solution

A routing engine where QPSO is the core global-search optimizer, refined by classical local search (2-opt), and applied to a real weighted road graph built from OpenStreetMap. The differentiating layer, VA-QPSO, adapts the exploration parameter β to measured traffic volatility. The system does not re-route on every small traffic fluctuation: it monitors external/background traffic, triggers re-optimization only when congestion or volatility crosses a defined threshold, and re-solves only the affected zone when possible. This keeps routes stable while still reacting quickly to genuinely disruptive conditions.

Traffic generated by the platform's own controlled fleet is kept separate from the traffic signal used for rerouting. In other words, if 20 vehicles use MARGA in the same area, they do not automatically make that area appear congested just because they are MARGA users. The same external traffic state may be observed by all vehicles in a zone, but each vehicle receives a route based on its own origin, destination, stop sequence, vehicle constraints, and the affected road network.

7. Product Goals

- Correctly implement and validate QPSO for VRP/shortest-path on a real road graph.
- Demonstrably reduce total travel time, distance, and congestion versus a naive baseline.
- Prove QPSO's competitiveness against OR-Tools and at least one other classical metaheuristic, with real benchmark numbers.
- Ship it as a working API and demo dashboard — not a notebook.

8. User Stories

- As a fleet operator, I want to input a depot and a list of delivery stops so that I get an optimized multi-vehicle route respecting capacity limits.
- As a fleet operator, I want the system to re-plan when traffic changes so that my drivers aren't stuck following a stale route.
- As a judge/evaluator, I want to see QPSO's performance compared side-by-side with OR-Tools/GA on standard benchmarks so that I can trust the “quantum-inspired” claim is real.
- As a developer extending this later, I want each solver behind a common interface so that I can swap in a new algorithm without touching the rest of the system.

9. Functional Requirements

ID	Requirement
FR-1	Ingest a real road network (OpenStreetMap via OSMnx) as a weighted directed graph
FR-2	Accept a routing problem: depot, stops, vehicle count/capacity, time windows
FR-3	Solve the problem using QPSO with a discrete route encoding (random-key)
FR-4	Solve the same problem using at least one classical baseline (OR-Tools) for comparison
FR-5	Handle constraints (capacity, time windows) via penalty/repair logic

ID	Requirement
FR-6	Apply local-search polish (2-opt) to QPSO's output
FR-7	Run and report benchmark comparisons (solution quality, runtime, convergence) on CVRPLIB/Solomon instances
FR-8	Expose functionality via a REST API (/optimize, /benchmark)
FR-9	Display results (route, cost, comparison chart) on a simple dashboard
FR-10	Measure traffic/volatility from external or background traffic data separately from the controlled MARGA fleet, so the platform does not create its own congestion signal.
FR-11	Trigger re-optimization only when a configurable congestion/volatility threshold is crossed, with a stability/cooldown rule to avoid repeated unnecessary rerouting.
FR-12	When disruption occurs, identify the affected zone/vehicles and re-optimize only the affected routes where feasible; keep unaffected routes unchanged.

10. Non-Functional Requirements

ID	Requirement
NFR-1	Solver architecture must be modular — any algorithm swappable behind one interface
NFR-2	Optimization for a 50–100 stop instance should return a result within a demo-friendly time (seconds, not minutes)
NFR-3	Benchmarks must be reproducible (fixed seeds, documented instance sources)
NFR-4	System deployable via Docker on a single instance
NFR-5	Code and architecture documented well enough for another engineer to extend it
NFR-6	Traffic updates must be isolated from the controlled fleet state; identical external traffic conditions must not force identical routes for vehicles with different trips.
NFR-7	Rerouting must be stable: small traffic fluctuations below the configured threshold should not trigger a new optimization.

11. Core Features (Must-Have)

- Real road graph ingestion (OSMnx)
- QPSO / VA-QPSO optimizer with discrete route encoding and constraint handling
- OR-Tools baseline for comparison
- Benchmark suite on CVRPLIB/Solomon instances with convergence charts
- REST API for optimization, re-optimization, traffic state, and benchmarking
- Partial re-optimization — freeze unaffected routes and re-solve only the affected zone/vehicles when feasible.
- Threshold-based adaptive rerouting — re-optimize only when traffic disruption is meaningful; do not react to every fluctuation.
- External/background traffic separation — the controlled fleet is not used to create the congestion signal that triggers rerouting.

12. Nice-to-Have Features

- Live SUMO simulation with injected incidents for a dynamic on-stage demo
- Traffic replay controls and scenario recording for a fully reproducible demo
- Partial re-optimization (freeze unaffected routes, re-solve only the affected zone)
- Interactive map dashboard with live route animation
- "Why this route" explainability panel

13. User Journey

Operator/Judge opens dashboard → selects a real road network and preset scenario → clicks "Optimize" → MARGA generates routes and shows the baseline comparison → traffic remains stable without unnecessary rerouting → a meaningful congestion/road incident is introduced → the system detects the affected zone → only affected vehicles are re-optimized while unaffected routes remain unchanged → before/after time, distance, congestion, and solver metrics are shown.

14. MVP Scope — What Must Work in the Demo

- A real road subgraph (one Bengaluru area) loaded and visualized, with a reproducible preset scenario.
- QPSO solving a CVRP instance with capacity constraints.
- OR-Tools solving the same instance as a baseline.
- A results view showing both routes and a comparison table (cost, runtime, gap to best-known where applicable).
- At least one CVRPLIB/Solomon benchmark run with a convergence chart, plus one dynamic traffic scenario showing threshold-based re-routing.
- A controlled-fleet traffic model in which background/external traffic drives congestion detection, while MARGA vehicles remain separate from that signal.
- If everything above works reliably, the PS's explicit requirements are fully met. Everything in Section 12 is a bonus, added only if time remains after the MVP is solid.

15. Out-of-Scope / Future Features

- Real quantum hardware execution (Qiskit/QAOA) — explicitly not the ask of this PS
- User accounts, authentication, multi-tenancy
- Mobile app
- Live paid traffic APIs (HERE/TomTom) — use simulated/background traffic for the hackathon; paid live feeds can be added later.
- Multi-depot, heterogeneous fleet, or other VRP variants beyond CVRP/VRPTW
- Kubernetes / multi-region deployment

16. Success Metrics

Metric	Target for Demo
Solution quality gap vs. best-known (CVRPLIB/Solomon)	Within a competitive range of OR-Tools/GA — reported honestly, not assumed
Runtime on 50–100 stop instance	Seconds, suitable for live demo
Reduction vs. naive/random baseline route	Clear, visible % improvement in distance/time
Reproducibility	Same benchmark instance and seed → same reported result every run
Rerouting stability	No re-optimization for traffic changes below the configured disruption threshold
Partial recovery time	Affected-zone re-optimization materially faster than full-fleet re-optimization in the demo scenario
Self-generated traffic isolation	Controlled MARGA vehicles do not directly inflate the external/background congestion signal

17. Risks and Assumptions

Risk / Assumption	Mitigation
QPSO may not beat OR-Tools on solution quality alone	State the honest claim: QPSO is competitive, and the hybrid + VA-QPSO layer is where the real advantage is expected, especially on large/dynamic instances — tested as a hypothesis, not assumed as a result
SUMO integration may take longer than expected	Treat it as a stretch feature; a simpler synthetic time-varying-edge-weight model is an acceptable fallback
Discrete route encoding (random-key) is the trickiest engineering piece	Build and test it early (Phase 2 of the build plan), not last
Judges may probe “is this really quantum”	Have the one-line honest answer ready: classical hardware, quantum-inspired search dynamics, architected to interface with real quantum/path-integral backends later
The platform may accidentally use its own fleet as the congestion signal	Maintain separate background/external traffic state and controlled-fleet state; only the former feeds congestion/volatility detection.
Frequent traffic fluctuations could cause route thrashing	Use a meaningful disruption threshold plus a cooldown/stability rule before another re-optimization is allowed.